

MADRIDISTA HUB

Ghid tehnic complet — Cum functioneaza proiectul

Acest document explica in detaliu arhitectura, fluxul de date si codul proiectului Madridista Hub, o aplicatie web full-stack construita cu ASP.NET Core MVC + Entity Framework Core. Este conceput ca ghid de prezentare — parcurge-l in paralel cu codul din Visual Studio.

Tehnologie	ASP.NET Core 8, Entity Framework Core, PostgreSQL
Pattern	Repository + Service Layer + DTOs + Dependency Injection
Frontend	Razor Views (Hybrid) + SPA component (LiveStats)
Autentificare	ASP.NET Core Identity cu roluri Admin si User
API	REST API la /api/players cu Swagger UI

1. Structura proiectului

Proiectul e organizat pe layere, fiecare cu o responsabilitate clara. Aceasta separare se numeste Clean Architecture si este un criteriu explicit de punctaj.

Folder	Ce contine
Models/	Entitatile EF Core — clasele care reprezinta tabelele din baza de date
DTOs/	Data Transfer Objects — ce trimitem/primim prin API si views, fara sa expunem entitatile direct
Repositories/	Acces la date — interfata + implementare pentru fiecare entitate
Services/	Logica business — interfata + implementare, foloseste repository-urile
Controllers/	MVC Controllers (Razor) + API Controllers (/api/*)
Views/	Paginile Razor (.cshtml) — HTML generat server-side
Data/	ApplicationDbContext + DataSeeder (seeding roluri si admin)
Middleware/	ExceptionMiddleware — gestionare globala a erorilor
ViewModels/	Modele pentru Login, Register, Profile, Error
Migrations/	Istoricul modificarilor schemei bazei de date (EF Core)

Regula de aur: un Controller nu vorbeste niciodata direct cu baza de date. Vorbeste cu un Service, care vorbeste cu un Repository, care vorbeste cu DbContext. Aceasta este separarea de responsabilitati (Separation of Concerns).

2. Entitatile si baza de date

Proiectul are 10 entitati EF Core (cerinta minima: 6, inclusiv User si Role din Identity). Fiecare entitate corespunde unui tabel in PostgreSQL.

Entitate	Fisier	Relatii
Team	Models/Team.cs	One-to-Many cu Player, Staff, Match
Player	Models/Player.cs	Many-to-One cu Team; Many-to-Many cu Match
Match	Models/Match.cs	Many-to-One cu Team; Many-to-Many cu Player
PlayerMatch	Models/PlayerMatch.cs	Tabela jonctiune Player-Match + MinutesPlayed, Rating
Staff	Models/Staff.cs	Many-to-One cu Team
Sponsor	Models/Sponsor.cs	Many-to-Many cu Team prin TeamSponsor
TeamSponsor	Models/TeamSponsor.cs	Tabela jonctiune Team-Sponsor
FavoritePlayer	Models/FavoritePlayer.cs	Leaga IdentityUser cu Player
WatchlistMatch	Models/WatchlistMatch.cs	Leaga IdentityUser cu Match
MatchComment	Models/MatchComment.cs	Many-to-One cu Match si IdentityUser
IdentityUser	(Identity)	Utilizatorul autentificat — gestionat de ASP.NET Core Identity
IdentityRole	(Identity)	Roluri: Admin si User

Relatii One-to-Many

O echipa (Team) are mai multi jucatori (Players). In cod:

```
// Models/Team.cs
public ICollection Players { get; set; } = new List();

// Models/Player.cs
public int TeamId { get; set; }           // cheia straina
public Team? Team { get; set; }          // navigation property
```

Relatii Many-to-Many

Un jucator poate juca in mai multe meciuri, un meci are mai multi jucatori. Se implementeaza printr-o tabela de jonctiune (PlayerMatch) care stocheaza si date aditionale:

```
// Models/PlayerMatch.cs
public int PlayerId { get; set; }
public Player? Player { get; set; }
public int MatchId { get; set; }
public Match? Match { get; set; }
public int MinutesPlayed { get; set; }
public double Rating { get; set; }
```

3. Fluxul unei cereri HTTP

Cand un utilizator deschide /Players/Index, iata ce se intampla pas cu pas:

Pas	Cine actioneaza	Ce se intampla
1	Browser	Trimite GET /Players/Index catre server
2	ASP.NET Router	Identifica PlayersController, actiunea Index()
3	PlayersController	Apeleaza _playerService.GetAllPlayersAsync()
4	PlayerService	Apeleaza _repository.GetAllAsync()
5	PlayerRepository	Executa SELECT pe baza de date prin EF Core
6	EF Core + PostgreSQL	Returneaza lista de entitati Player
7	PlayerService	Mapeaza entitatile la PlayerReadDto
8	PlayersController	Primeste IEnumerable<PlayerReadDto>, apeleaza View(players)
9	Razor View	Genereaza HTML cu datele din DTO
10	Browser	Afiseaza pagina utilizatorului

De ce DTOs si nu entitatile direct?

Daca ai returna entitatea Player direct din controller, ai expune si cheia straina TeamId, navigation properties (Team, PlayerMatches). DTO-ul (PlayerReadDto) contine doar ce ai nevoie: Name, Position, Goals, Assists, TeamName. Aceasta se numeste **encapsulare** si protejeaza structura interna a bazei de date.

4. Autentificare si autorizare

Proiectul foloseste ASP.NET Core Identity — un sistem complet integrat in framework care gestioneaza utilizatori, parole (hash bcrypt), sesiuni si roluri.

Cum functioneaza login-ul

1. Utilizatorul completeaza formularul (Views/Account/Login.cshtml)
2. Datele ajung la AccountController.Login (POST)
3. Controllerul apeleaza _signInManager.PasswordSignInAsync()
4. Daca reuseste, ASP.NET creeaza un **cookie de sesiune criptat** in browser
5. La fiecare cerere urmatoare, browser-ul trimite automat cookie-ul

Roluri: Admin si User

La pornirea aplicatiei, DataSeeder.cs creeaza automat rolurile si un cont admin default. Endpoint-urile sensibile sunt protejate cu:

```
[Authorize(Roles = "Admin")]
public async Task Create() { ... }

// Un User obisnuit care acceseaza /Players/Create
// este redirectionat automat la /Account/AccessDenied (403)
```

Fisier	Rol
Data/DataSeeder.cs	Creeaza rolurile Admin/User si contul admin la start
Controllers/AccountController.cs	Login, Register, Logout
Views/Account/Login.cshtml	Formularul de autentificare
Program.cs	Configurare Identity + cookie paths

5. REST API si Swagger

Pe langa paginile Razor, proiectul expune un REST API la /api/players. Acesta este consumat de componenta SPA (LiveStats) si poate fi testat cu Swagger UI la <http://localhost:5090/swagger>.

Endpoint	Status Code	Descriere
GET /api/players	200 OK	Returneaza toti jucatorii ca JSON
GET /api/players/{id}	200 / 404	Un jucator dupa ID sau 404 daca nu exista
POST /api/players	201 Created + Location	Creeaza jucator nou; returneaza Location header
PUT /api/players/{id}	204 / 404	Actualizeaza jucator existent
DELETE /api/players/{id}	204 / 404	Sterge jucator (necesita rol Admin)

De ce 201 + Location header la POST?

Standardul REST spune ca la crearea unui resource nou, serverul returneaza 201 Created cu header-ul Location care indica URL-ul noului resource:

```
var createdId = await _playerService.CreatePlayerAsync(dto);
var createdPlayer = await _playerService.GetPlayerByIdAsync(createdId);
return CreatedAtRoute("GetPlayerById", new { id = createdId }, createdPlayer);
// Raspuns: 201 Created
// Header: Location: /api/players/42
```

De ce 204 la DELETE si PUT?

204 No Content insemna: operatia a reusit, dar nu am nimic de returnat. E mai eficient decat 200 cu body gol. La stergere sau update nu ai nevoie sa returnezi intreaga entitate.

6. Componenta SPA — LiveStats

LiveStats (Views/Home/LiveStats.cshtml) bifaza cerinta de 'minimum o pagina SPA care consuma /api/*'. Nu foloseste Razor pentru date — JavaScript face un fetch direct catre API si construiește HTML-ul dinamic.

Fluxul paginii LiveStats

Pas	Ce se intampla
1	Pagina se incarca — HTML-ul vine de la server (Razor), dar fara date
2	JavaScript asculta evenimentul DOMContentLoaded
3	Se afiseaza un spinner de incarcare
4	fetch('/api/players') trimite o cerere GET asincron catre API
5	Raspunsul JSON soseste
6	JavaScript construiește dinamic randurile tabelului si le injecteaza in DOM
7	Spinnerul dispare, tabelul apare — fara reincarcarea paginii

```
fetch('/api/players')
  .then(response => {
    if (!response.ok) throw new Error('API error');
    return response.json();
  })
  .then(data => {
    spinner.classList.add('d-none');
    data.forEach(player => {
      tableBody.innerHTML += `${player.name}...`;
    });
  })
  .catch(error => { errorMsg.classList.remove('d-none'); });
```

Razor (server-side)	SPA / fetch (client-side)
HTML generat pe server inainte sa ajunga la browser	Pagina e incarcata o singura data, datele vin asincron
La fiecare actiune, pagina se reimprospateaza	Datele se actualizeaza fara reincarcarea paginii
Cod C# in .cshtml	Cod JavaScript — fetch, .then(), DOM manipulation

7. Exception Middleware si Logging

ExceptionHandlerMiddleware.cs

In loc sa lasi fiecare controller sa trateze erorile individual, middleware-ul global prinde orice exceptie neasteptata si returneaza un JSON cu status code corect. E inregistrat primul in pipeline:

```
// Program.cs
app.UseExceptionHandler();

// Raspuns automat pentru orice eroare neasteptata:
// {
//   "statusCode": 500,
//   "message": "An unexpected error occurred."
// }
```

ILogger in Service Layer

Fiecare service primeste prin injectie un ILogger si logheaza operatiile. Log-urile apar in consola la dotnet run:

```
public class PlayerService : IPlayerService
{
    private readonly ILogger _logger;

    public async Task<T> GetAllPlayersAsync()
    {
        _logger.LogInformation("Fetching all players.");
        // ...
    }

    public async Task GetPlayerByIdAsync(int id)
    {
        var player = await _repository.GetByIdAsync(id);
        if (player == null)
            _logger.LogWarning("Player with ID {Id} not found.", id);
        // ...
    }
}
```


8. Dependency Injection

Dependency Injection este mecanismul prin care ASP.NET Core creeaza si gestioneaza instantele claselor. Nu scriem niciodata 'new PlayerService()' manual — framework-ul se ocupa. Tot ce facem e sa inregistram serviciile in Program.cs:

```
// Program.cs
builder.Services.AddScoped();
builder.Services.AddScoped();
// 'Scoped' = o instanta noua per cerere HTTP
```

Cand ASP.NET creeaza un PlayersController, vede ca are nevoie de IPlayerService. Il creeaza automat. IPlayerService are nevoie de IPlayerRepository — il creeaza si pe ala. Totul e rezolvat automat in lant.

Avantaj major: daca vrei sa schimbi baza de date (ex. de la PostgreSQL la MySQL), schimbi doar implementarea repository-ului. Controllerele si serviciile raman neatinse.

9. Intrebari frecvente la prezentare

Acestea sunt intrebarile pe care un profesor le pune de obicei la o prezentare DAW.

Q: Ce este Repository Pattern si de ce l-ai folosit?

A: Repository Pattern abstractizeaza accesul la date. In loc ca serviciile sa stie direct ca folosesc EF Core si PostgreSQL, ele cer date printr-o interfata (IPlayerRepository). Daca schimbi baza de date, schimbi doar implementarea repository-ului, nu toata logica business.

Q: Ce este un DTO si de ce nu expui entitatile direct?

A: DTO (Data Transfer Object) e o clasa simpla cu doar datele necesare interfetei. Entitatea Player are navigation properties si chei straine — daca o expui direct, expui structura interna a bazei de date. PlayerReadDto contine doar Name, Position, Goals, TeamName — exact ce afiseaza UI-ul.

Q: Cum functioneaza autentificarea?

A: Folosesc ASP.NET Core Identity. La login, PasswordSignInAsync verifica parola (hash bcrypt automat), creeaza un cookie de sesiune criptat. La fiecare cerere urmatoare, browser-ul trimite cookie-ul si ASP.NET stie cine esti si ce roluri ai.

Q: Cum protejezi endpoint-urile de Admin?

A: Cu atributul [Authorize(Roles = 'Admin')] pe controller sau pe actiune. Daca un utilizator fara rolul Admin incearca sa acceseze, este redirectionat automat la /Account/AccessDenied (403 Forbidden).

Q: Ce face ExceptionMiddleware?

A: Prinde orice exceptie neateptata din intreaga aplicatie si returneaza un JSON cu statusCode si message, in loc sa afiseze stack trace-ul utilizatorului. E inregistrat primul in pipeline (app.UseMiddleware).

Q: De ce 201 si Location header la POST /api/players?

A: Standardul REST HTTP spune ca la crearea unui resource nou, serverul returneaza 201 Created cu header-ul Location care indica URL-ul noului resource. CreatedAtRoute face asta automat.

Q: Ce este Dependency Injection?

A: Un mecanism prin care ASP.NET Core creeaza si injecteaza automat dependentele claselor. Nu scriem niciodata new PlayerService() — inregistram interfata cu implementarea in Program.cs si framework-ul se ocupa de instantiere. Avantaj: testabilitate si decuplare.

Q: Care e diferenta dintre LiveStats si restul paginilor?

A: Restul paginilor sunt Razor — HTML-ul e generat server-side. LiveStats e o componenta SPA — pagina e incarcata fara date, iar JavaScript face fetch catre /api/players si construiesc tabelul dinamic in browser, fara reincarcarea paginii.

Q: Ce inseamna One-to-Many si Many-to-Many in proiectul tau?

A: One-to-Many: o echipa (Team) are mai multi jucatori (Players) — TeamId e cheie straina in Players. Many-to-Many: un jucator poate juca in mai multe meciuri si un meci are mai multi jucatori — implementat prin tabela PlayerMatch care stocheaza si MinutesPlayed si Rating.

Q: De ce ai folosit interfete (IPlayerService) in loc de clase concrete?

A: Interfetele permit schimbarea implementarii fara sa modifichi codul care le foloseste. Permite si testare unitara (mock-uri). Este principiul Dependency Inversion din SOLID.

10. Checklist cerinte proiect

Verifica rapid ce ai bifat inainte de predare.

Cerinta	Pct	Status	Fisier principal
ASP.NET Core Web API RESTful	5p	OK	Controllers/PlayersApiController.cs
Minimum 5 controllere	5p	OK (9)	Controllers/
Status codes corecte (200,201,204,400,404)	5p	OK	PlayersApiController.cs
EF Core, minimum 6 entitati	10p	OK (10)	Models/
Relatii One-to-Many	10p	OK	Models/Team.cs, Player.cs
Relatii Many-to-Many	10p	OK	Models/PlayerMatch.cs, TeamSponsor.cs
Migrations EF Core	10p	OK	Migrations/
ASP.NET Core Identity	5p	OK	Program.cs, AccountController.cs
Roluri Admin + User	5p	OK	Data/DataSeeder.cs
Minimum 2 endpoint-uri Admin-only	5p	OK	[Authorize(Roles=Admin)] in controllere
Exception Middleware global	3p	OK	Middleware/ExceptionMiddleware.cs
Logging ILogger<T>	2p	OK	Services/*.cs
DTOs (fara expunere EF directa)	5p	OK	DTOs/
Service Layer	5p	OK	Services/
Repository Pattern	5p	OK	Repositories/
Dependency Injection	5p	OK	Program.cs
Minimum 5 pagini	5p	OK (13+)	Views/
Login + Register functional	4p	OK	Views/Account/, AccountController.cs
Pagini protejate (redirect login)	2p	OK	[Authorize] pe controllere
Formulare cu validare client+server	2p	OK	Views/**/Create.cshtml
API integration SPA	2p	OK	Views/Home/LiveStats.cshtml
Structura proiect clara	5p	OK	Controllers/Services/Repositories/Views